

Practice Interview

Objective

The partner assignment aims to provide participants with the opportunity to practice coding in an interview context. You will analyze your partner's Assignment 1. Moreover, code reviews are common practice in a software development team. This assignment should give you a taste of the code review process.

Group Size

Each group should have 2 people. You will be assigned a partner

Parts:

- Part 1: Complete 1 of 3 questions
- Part 2: Review your partner's Assignment 1 submission
- Part 3: Perform code review of your partner's assignment 1 by answering the questions below
- Part 3: Reflect on Assignment 1 and Assignment 2

Part 1:

You will be assigned one of three problems based on your first name. Enter your first name, in all lower case, execute the code below, and that will tell you your assigned problem. Include the output as part of your submission (do not clear the output). The problems are based-off problems from Leetcode.

```
import hashlib

def hash_to_range(input_string: str) -> int:
    hash_object = hashlib.sha256(input_string.encode())
    hash_int = int(hash_object.hexdigest(), 16)
    return (hash_int % 3) + 1

input_string = "augustina" # Partner name: augustina
result = hash_to_range(input_string)
print(result)

3
```

Starter Code for Question 1

```
# Definition for a binary tree node.
# class TreeNode(object):
#     def __init__(self, val = 0, left = None, right = None):
#         self.val = val
```

```

#         self.left = left
#         self.right = right
def is_duplicate(root: TreeNode) -> int:
    if root is None:
        return -1

    # set to store values we have already seen
    seen = set()

    # queue for BFS traversal (level-order)
    queue = [root]

    while queue:
        # get the first node in the queue
        node = queue.pop(0)

        # if we have already seen this value, it's a duplicate
        if node.val in seen:
            return node.val

        # mark current value as seen
        seen.add(node.val)

        # add left child to queue if it exists
        if node.left:
            queue.append(node.left)

        # add right child to queue if it exists
        if node.right:
            queue.append(node.right)

    # if no duplicate found, return -1
    return -1

## What traversal method is this? It is Level-order traversal.

```

Starter Code for Question 2

```

# Definition for a binary tree node.
# class TreeNode(object):
#     def __init__(self, val = 0, left = None, right = None):
#         self.val = val
#         self.left = left
#         self.right = right
def bt_path(root: TreeNode) -> List[List[int]]:
    result = []

    def dfs(node, path):
        if node is None:
            return

```

```

    # add current node to path
    path.append(node.val)

    # if leaf node, save the path
    if node.left is None and node.right is None:
        result.append(list(path)) # copy current path
    else:
        # explore left and right subtrees
        dfs(node.left, path)
        dfs(node.right, path)

    # backtrack (remove last node before returning)
    path.pop()

dfs(root, [])
return result

```

Starter Code for Question 3

```

def missing_num(nums: List) -> int:
    n = len(nums)

    # mark seen numbers using a set
    seen = set(nums)

    # collect missing numbers from 1 to n
    missing = []

    for i in range(1, n + 1):
        if i not in seen:
            missing.append(i)

    # if nothing is missing, return -1
    if len(missing) == 0:
        return -1

    return missing

```

Part 2:

You and your partner must share each other's Assignment 1 submission.

https://github.com/augustina2023/algorithms_and_data_structures/pulls

Part 3:

Create a Jupyter Notebook, create 6 of the following headings, and complete the following for your partner's assignment 1:

- Paraphrase the problem in your own words.

My partner augustina is being assigned Q3.

We are given a list of numbers and need to rearrange it so that every zero in the list is moved to the end. At the same time, all non-zero numbers must keep the same relative order they originally had. We are not allowed to remove or change any values, only shift their positions so that all zeros end up grouped at the back of the list.

- Create 1 new example that demonstrates you understand the problem.
Trace/walkthrough 1 example that your partner made and explain it.

New Example:

Input: [3, 0, 1, 0, 2, 0, 5]

Output: [3, 1, 2, 5, 0, 0, 0]

All non-zero numbers (3, 1, 2, 5) stay in the same order, and all zeros are moved to the end.

Trace my parner example:

Input: [0, 0, 6, 7, 0, 5, 4]

Output: [6, 7, 5, 4, 0, 0, 0]

We are using a bubble-sort style approach. The outer loop = number of full passes through the list, and the inner loop = compares adjacent elements. Swap happens when a 0 is on the left and a non-zero is on the right

- Copy the solution your partner wrote.

```
def move_zeros(nums):  
    i = 0  
    # first loop for repitition  
    for i in range(len(nums)):  
  
        # second loop for comparison  
        for i in range(len(nums)-1):  
  
            # switch the non-zero int with zero  
            if nums[i] == 0 and nums[i+1] != 0:  
                nums[i], nums[i+1] = nums[i+1], nums[i]  
  
        i += 1  
  
    return nums
```

The solutuon I chose is bubble sort. It works by comparing 2 neighbouring elements in the list, then switching the positions of the element if one of them is a zero. This results in all the zeros being at the end of the list.

This is has a time complexity of $O(n^2)$. It is a bubble search with

a nested loop.

Since a list is the data structure that is storing the data, the space complexity is $O(n)$.

An alternative solution could be to perform an insertion sort algorithm. This algorithm compares the current element in the list with the preceeding one, then swaps them, moving the element of greater value up the list (to the right)

- Explain why their solution works in your own words.

The algorithm works by repeatedly going through the list and comparing neighbouring elements. When a zero is found in front of a non-zero number, they get swapped.

Each full pass through the list (outer loop) helps push the zeros more towards the right side of the list, while the non-zero numbers slowly move towards the front in the correct order.

- Explain the problem's time and space complexity in your own words.

My partner explained the time and space complexity. The reason the time complexity is $O(n^2)$ is because the solution uses a nested loop, where the list is repeatedly scanned and each element is compared multiple times.

The space complexity is $O(1)$ because the algorithm does not create an additional list to store the data. It only modifies the original list in place and uses a small number of extra variables.

- Critique your partner's solution, including explanation, and if there is anything that should be adjusted.

The solution works correctly, but the time complexity of $O(n^2)$ is not ideal. In most cases, we aim for a more efficient solution such as $O(n)$ or $O(1)$ if possible. As the input size increases, the runtime of an $O(n^2)$ algorithm grows much faster, which can lead to slower performance and higher resource usage.

The bubble sort approach does solve the problem and produces the correct output, so it is a valid solution. However, because it uses a nested loop, it is not efficient for larger datasets.

If we can avoid using nested loops, the time complexity could be improved significantly. For example, there are alternative approaches (such as a two-pointer method) that can solve this problem in $O(n)$ time.

Overall, while the current solution is correct and easy to

understand, there are more efficient methods that would be better in terms of performance.

Part 4:

Please write a 200 word reflection documenting your process from assignment 1, and your presentation and review experience with your partner at the bottom of the Jupyter Notebook under a new heading "Reflection." Again, export this Notebook as pdf.

Reflection

Assignment 1 required me to learn about algorithmic problems and translate them into Python solutions. In particular, I had to analyze every question, especially about handling the input data structure – lists and binary trees, etc. Moreover, I learned how essential it is to understand the problem before implementing the solution in code. For example, in the "Move Zeros" problem, I focused on the correctness of the code at first, but then I understood the importance of efficiency and time complexity.

In the process of coding, I was able to try several strategies including using nested loops and applying more optimized techniques such as the two-pointer technique. In this way, I was able to see how efficiency affects the solution, whether it is inefficient but straightforward solution $O(n^2)$ or an efficient one $O(n)$. Moreover, I became better at describing my logic in my own words.

It was useful to have a partner because it helped me compare the advantages and drawbacks of different approaches. During the code review, I learned how to critically evaluate, translate and comment the code.

Overall, this assignment helped me improve both my problem-solving skills and my ability to communicate technical ideas more clearly.

Evaluation Criteria

We are looking for the similar points as Assignment 1

- Problem is accurately stated
- New example is correct and easily understandable
- Correctness, time, and space complexity of the coding solution
- Clarity in explaining why the solution works, its time and space complexity
- Quality of critique of your partner's assignment, if necessary

Submission Information

📖 Please review our [Assignment Submission Guide](#) 📖 for detailed instructions on how to format, branch, and submit your work. Following these guidelines is crucial for your submissions to be evaluated correctly.

Submission Parameters:

- Submission Due Date: HH:MM AM/PM - DD/MM/YYYY
- The branch name for your repo should be: `assignment-2`
- What to submit for this assignment:
 - This Jupyter Notebook (assignment_2.ipynb) should be populated and should be the only change in your pull request.
- What the pull request link should look like for this assignment:
`https://github.com/<your_github_username>/algorithms_and_data_structures/pull/<pr_id>`
 - Open a private window in your browser. Copy and paste the link to your pull request into the address bar. Make sure you can see your pull request properly. This helps the technical facilitator and learning support staff review your submission easily.

Checklist:

- ☐ Created a branch with the correct naming convention.
- ☐ Ensured that the repository is public.
- ☐ Reviewed the PR description guidelines and adhered to them.
- ☐ Verify that the link is accessible in a private browser window.

If you encounter any difficulties or have questions, please don't hesitate to reach out to our team via our Slack at `#cohort-6-help`. Our Technical Facilitators and Learning Support staff are here to help you navigate any challenges.

